

Computational Analysis of Power Systems

Power Flow, Fault Analysis & Sensitivity Analysis Simulator

Project Documentation – ECE 2774 – Spring 2026

Instructors	: Dr. Robert Kerestes
	: Dr. Paulo Radatz
Teaching Assistant	: Mr. Collin Griffin
Student	: Trinh Truong

1. Project Overview

This project implements a complete power system analysis simulator in Python, capable of performing:

- Newton-Raphson power flow analysis
- Three-phase symmetrical fault analysis
- Linear voltage sensitivity analysis

The simulator is built from scratch using object-oriented programming, with each component of the power system modeled as a Python class. The system is validated against PowerWorld Simulator using a 7-bus looped transmission system.

2. File Structure

File	Class	Purpose
Bus.py	Bus	Represents a power system bus node
Transformer.py	Transformer	Models transformer with series impedance
TransmissionLine.py	TransmissionLine	Models transmission line using pi-model
Load.py	Load	Represents constant power load at a bus
Generator.py	Generator	Represents generator with voltage/power setpoints
Settings.py	Settings	System-wide per-unit base parameters
Circuit.py	Circuit	Main container: assembles network, builds Ybus, computes power flow equations
Jacobian.py	Jacobian	Builds the Newton-Raphson Jacobian

		matrix
NRSolver.py	NRSolver	Newton-Raphson iterative solver and fault analysis
SensAna.py	SensAna	Linear voltage sensitivity analysis using J inverse
SevenBus.py	-	Validation script: 7-bus power flow
Fault_Validation.py	-	Validation script: 7-bus fault analysis
Sensitivity_Analysis.py	-	Validation script: sensitivity analysis

3. Class Descriptions

3.1. Bus Class

Represents a node in the power system where equipment connects.

Attribute	Type	Description
name	str	Unique identifier for the bus
nominal_kV	float	Physical voltage level in kilovolts
bus_type	str	Bus classification: Slack, PQ, or PV
vpu	float	Per-unit voltage magnitude (initialized to 1.0)
delta	float	Voltage angle in radians (initialized to 0.0)
bus_index	int	Auto-assigned unique integer index (class-level counter)

Bus Types:

- Slack: Reference bus, V and δ fixed, P and Q generated to balance system
- PQ: Load bus, P and Q specified, V and δ solved by Newton-Raphson
- PV: Generator bus, P and V specified, Q and δ solved by Newton-Raphson

3.2. Transformer Class

Models a transformer as a series impedance element connecting two buses.

Attribute	Type	Description
name	str	Transformer identifier

bus1_name/bus2_name	str	Names of connected buses
r	float	Series resistance in per-unit (system base)
x	float	Series reactance in per-unit (system base)
yseries	complex	Series admittance: $1/(r + jx)$

Method: calc_yprim()

Returns the 2×2 primitive admittance matrix as a labeled pandas DataFrame:

$$\begin{bmatrix} y & -y \end{bmatrix}$$

$$\begin{bmatrix} -y & y \end{bmatrix}$$

Where $y = y_{\text{series}}$. Diagonal entries are $+y$ (self-admittance), off-diagonals are $-y$ (mutual admittance).

Transformers modeled as series-only elements (no shunt admittance).

3.3. TransmissionLine Class

Models a transmission line using the pi-model, which includes both series impedance and shunt admittance to ground at each end.

Attribute	Type	Description
name	str	Line identifier
bus1_name/bus2_name	str	Names of connected buses
r	float	Series resistance per-unit
x	float	Series reactance per-unit
g	float	Shunt conductance per-unit
b	float	Shunt susceptance per-unit
yseries	complex	Series admittance: $1/(r + jx)$
yshunt	complex	Shunt admittance: $g + jb$

Method: calc_yprim()

Returns the 2×2 pi-model primitive admittance matrix:

$$\begin{bmatrix} y + y_s/2 & -y \end{bmatrix}$$

$$\begin{bmatrix} -y & y + y_s/2 \end{bmatrix}$$

The $y_s/2$ term on each diagonal represents half the shunt admittance sitting at each end of the line (pi-model).

3.4. Load Class

Represents a constant real and reactive power load connected to a bus.

Attribute	Type	Description
name	str	Load identifier
bus1_name	str	Name of connected bus
mw	float	Real power consumption in MW
mvar	float	Reactive power consumption in Mvar
p	float	Per-unit real power (set by <code>calc_p()</code>)
q	float	Per-unit reactive power (set by <code>calc_q()</code>)

Methods:

- `calc_p(settings)`: Converts MW to per-unit by dividing by `settings.sbase`
- `calc_q(settings)`: Converts Mvar to per-unit by dividing by `settings.sbase`

3.5. Generator Class

Represents a generator connected to a bus with specified operating setpoints.

Attribute	Type	Description
name	str	Generator identifier
bus1_name	str	Name of connected bus
voltage_setpoint	float	Terminal voltage setpoint in per-unit
mw_setpoint	float	Real power output setpoint in MW
p	float	Per-unit real power (set by <code>calc_p()</code>)
xpp	float (optional)	Subtransient reactance for fault analysis
mva	float (optional)	Generator MVA rating for base conversion

Method: `calc_p(settings)`

Converts MW setpoint to per-unit. Generators have no q attribute since Q is a free variable (not specified) for PV and Slack buses.

3.6. Settings Class

Centralized class that defines system-wide per-unit base parameters.

Attribute	Type	Description
freq	60 Hz	System frequency
sbase	100 MVA	System base apparent power

3.7. Circuit Class

The main container class that assembles the complete power system network. It manages all equipment objects and provides methods to build network matrices and compute power flow equations.

Key Attributes:

- buses, transformers, transmission_lines, generators, loads: Dictionaries storing equipment objects (key = name string, value = object)
- ybus: Pandas DataFrame storing the nodal admittance matrix
- zbus: Pandas DataFrame storing the bus impedance matrix (fault analysis)
- settings: Settings object with system base parameters

Add Methods:

Each add method creates an equipment object and stores it in the appropriate dictionary. Duplicate names raise a ValueError.

- circuit.add_bus(name, nominal_kv, bus_type)
- circuit.add_transformer(name, bus1_name, bus2_name, r, x)
- circuit.add_transmission_line(name, bus1_name, bus2_name, r, x, g, b)
- circuit.add_generator(name, bus1_name, voltage_setpoint, mw_setpoint, xpp=None, mva=None)
- circuit.add_load(name, bus1_name, mw, mvar)

Method: calc_ybus()

Builds the $N \times N$ nodal admittance matrix by stamping each element's primitive admittance matrix (Yprim) into the correct positions. Uses a bus_map dictionary to translate bus names to matrix indices. Result stored as self.ybus (labeled DataFrame).

Method: calc_ybus_fault()

Returns a modified Ybus for fault analysis by adding generator subtransient admittances ($1/jx''$) to the diagonal entries at each generator bus. Converts xpp from machine base to system base using:

$$xpp_{sys} = xpp * (sbase / gen.mva)$$

Method: calc_zbus()

Computes the bus impedance matrix by inverting the faulted Ybus:

$$Zbus = inv(Ybus_fault)$$

Result stored as self.zbus (labeled DataFrame).

Method: compute_power_injection(bus, ybus, voltages)

Computes real and reactive power injections at a bus using:

$$P_i = |V_i| * \sum_j |V_j| * (G_{ij} * \cos(d_{ij}) + B_{ij} * \sin(d_{ij}))$$

$$Q_i = |V_i| * \sum_j |V_j| * (G_{ij} * \sin(d_{ij}) - B_{ij} * \cos(d_{ij}))$$

Returns Pi and Qi for PQ/Slack buses; returns Pi and None for PV buses (Q not specified).

Method: compute_power_mismatch(buses, ybus, voltages)

Builds the mismatch vector for Newton-Raphson. For each non-slack bus: computes P_spec and Q_spec from generators and loads, then subtracts calculated Pi and Qi. Returns numpy array ordered as $[\Delta P_all, \Delta Q_PQ]$ — all ΔP first then all ΔQ , matching the Jacobian structure.

3.8. Jacobian Class

Builds the Newton-Raphson Jacobian matrix, which is the matrix of first-order partial derivatives that linearizes the nonlinear power flow equations around an operating point.

Structure:

The Jacobian is partitioned into four submatrices of dimension $(2N-2-NPV) \times (2N-2-NPV)$:

$$J = \begin{bmatrix} J1 & J2 \\ J3 & J4 \end{bmatrix} = \begin{bmatrix} dP/d\delta & dP/dV \\ dQ/d\delta & dQ/dV \end{bmatrix}$$

Off-diagonal formulas ($k \neq n$):

$$J1_kn = V_k * |Y_{kn}| * V_n * \sin(d_k - d_n - \theta_{kn})$$

$$J2_kn = V_k * |Y_{kn}| * \cos(d_k - d_n - \theta_{kn})$$

$$J3_kn = -V_k * |Y_{kn}| * V_n * \cos(dk - dn - \theta_{kn})$$

$$J4_kn = V_k * |Y_{kn}| * \sin(dk - dn - \theta_{kn})$$

Diagonal formulas (k = n):

$$J1_kk = -V_k * \sum_{m \neq k} |Y_{km}| * V_m * \sin(dk - dm - \theta_{km})$$

$$J2_kk = \sum_m |Y_{km}| * V_m * \cos(dk - dm - \theta_{km}) + V_k * |Y_{kk}| * \cos(\theta_{kk})$$

$$J3_kk = V_k * \sum_{m \neq k} |Y_{km}| * V_m * \cos(dk - dm - \theta_{km})$$

$$J4_kk = \sum_m |Y_{km}| * V_m * \sin(dk - dm - \theta_{km}) - V_k * |Y_{kk}| * \sin(\theta_{kk})$$

Key Implementation Note — Diagonal Elements:

For J2 and J4 diagonal elements, V_k appears twice in the power equation (once as a standalone factor and once inside the summation self-term). The diagonal uses a full sum over ALL buses plus one extra self-term to correctly account for this double contribution.

3.9. NRSolver Class

Implements the Newton-Raphson iterative solver and fault analysis. Supports two modes: 'powerflow' and 'fault'.

Newton-Raphson Algorithm:

1. Initialize flat start: all $V=1.0$ pu, all $\delta=0.0$ radians, PV buses set to voltage_setpoint.
2. Build voltages snapshot dictionary from current bus states.
3. Compute power mismatch vector $f = [\Delta P, \Delta Q]$.
4. Check convergence: if $\max(|f|) < 0.001$, stop and print results.
5. Compute Jacobian J at current operating point.
6. Solve linear system: $J \times \Delta x = f$ using `np.linalg.solve()`.
7. Split Δx : first n_P entries = $\Delta \delta$ (angle corrections), remaining = ΔV (voltage corrections).
8. Update angles for all P_buses: $\delta += \Delta \delta$.
9. Update voltages for Q_buses only: $V += \Delta V$.
10. Repeat from Step 2 (maximum 50 iterations).

Fault Analysis Algorithm:

1. Validate faulted bus exists in circuit.
2. Rebuild Ybus and compute faulted Zbus = inv(Ybus_fault).
3. Calculate fault current: $I_{\text{fault}} = V_f / Z_{nn}$ (where Z_{nn} is diagonal Zbus entry at faulted bus).
4. Calculate post-fault voltages: $V_k = 1 - (Z_{kn} / Z_{nn}) \times V_f$ for all buses k.
5. Faulted bus voltage collapses to 0.0 pu (since $Z_{nn}/Z_{nn} = 1$).

3.10. SensAna Class

Implements linear voltage sensitivity analysis using the converged Jacobian inverse. Enables fast prediction of voltage and angle changes due to small load perturbations without re-solving the full nonlinear power flow.

Theory:

At the converged operating point, the Jacobian provides a linear approximation:

$$\Delta x = J^{-1} * \Delta f$$

Where $\Delta f = [\Delta P, \Delta Q]$ is the power perturbation vector and $\Delta x = [\Delta \delta, \Delta V]$ is the resulting state change.

Sensitivity Matrices extracted from J^{-1} :

Matrix	Dimension	Physical Meaning
dDelta_dP	$n_P \times n_P$	How angles change with P perturbations
dDelta_dQ	$n_P \times n_Q$	How angles change with Q perturbations
dV_dP	$n_Q \times n_P$	How voltages change with P perturbations
dV_dQ	$n_Q \times n_Q$	How voltages change with Q perturbations

Unit Note: Python sensitivity matrices are in pu/pu. Divide by sbase (100 MVA) to compare with PowerWorld's per MW/Mvar values.

Method: perturb_bus(bus_name, delta_P=0.0, delta_Q=0.0)

Applies a small perturbation at a specified bus and returns the post-perturbation voltages and angles at

all buses. Validates that:

- Slack buses cannot be perturbed (P and Q are free variables)
- PV buses cannot have Q perturbations (voltage is fixed)

4. How to Run

4.1. Running Power Flow

Run the Python file:

```
SevenBus.py
```

This script will:

- Build the 7-bus system with all components
- Compute and print the full Ybus matrix
- Run Newton-Raphson power flow
- Print converged bus voltages and angles in degrees

Outputs:

```
The problem is converged after 4 iterations
Bus 1: vpu=1.0000, delta= 0.0000 deg (Slack)
Bus 2: vpu=0.9744, delta=-4.4027 deg (PQ)
Bus 3: vpu=0.9668, delta=-5.5168 deg (PQ)
Bus 4: vpu=0.9714, delta=-4.5789 deg (PQ)
Bus 5: vpu=0.9684, delta=-5.1480 deg (PQ)
Bus 6: vpu=0.9766, delta=-3.8369 deg (PQ)
Bus 7: vpu=1.0000, delta= 2.2186 deg (PV)
```

4.2. Running Fault Analysis

Run the Python file:

```
Fault_Validation.py
```

This script will:

- Build faulted Ybus (with generator subtransient reactances added to diagonals)
- Compute and print the Zbus matrix
- Run three-phase bolted fault analysis at every bus
- Print fault current (pu and kA) and post-fault bus voltages for each fault location

4.3. Running Sensitivity Analysis

Run the Python file:

`Sensitivity_Analysis.py`

This script will:

- Run base case power flow first
- Compute J inverse and extract all four sensitivity matrices (dV/dP , dV/dQ , $d\delta/dP$, $d\delta/dQ$)
- Print all sensitivity matrices with bus labels
- Apply +0.01 pu perturbations at Bus 3, Bus 4, and Bus 5 and show post-perturbation voltages and angles
- Validate linear prediction against full Newton-Raphson re-solve

5. Seven Bus System Test

5.1. PowerWorld Set Up

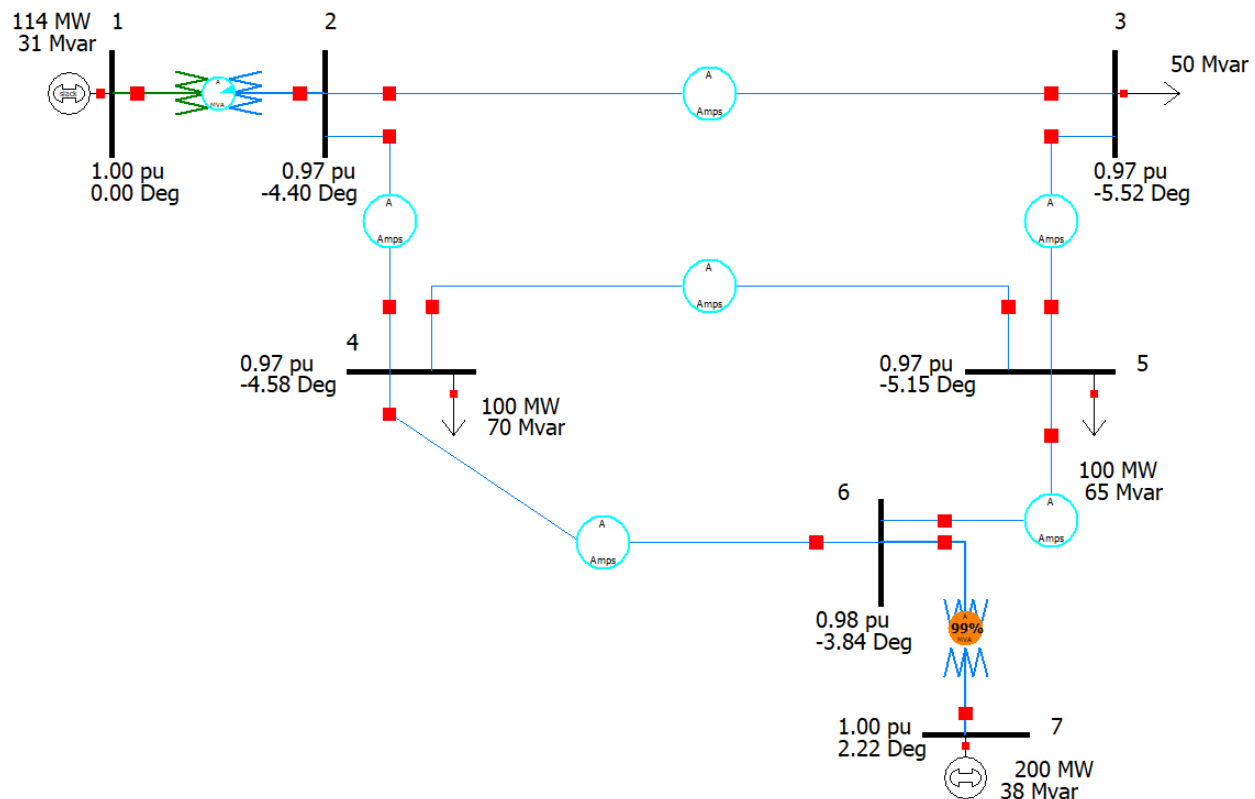


Figure 1. Distribution Results of the System in PowerWorld

5.2. Simulator and PowerWorld Results Comparison

- **Power Flow results:**

Bus	Type	Python Simulator		PowerWorld		Error	
		V (pu)	δ (deg)	V (pu)	δ (deg)	$ \Delta V $ (pu)	$ \Delta \delta $ (deg)
1	Slack	1.00000	0.0000	1.00000	0.0000	0.00000	0.0000
2	PQ	0.97444	-4.4027	0.97444	-4.4000	0.00000	0.0027
3	PQ	0.96682	-5.5168	0.96682	-5.5200	0.00000	0.0032
4	PQ	0.97137	-4.5789	0.97137	-4.5800	0.00000	0.0011
5	PQ	0.96839	-5.1480	0.96839	-5.1500	0.00000	0.0020
6	PQ	0.97661	-3.8369	0.97661	-3.8400	0.00000	0.0031
7	PV	1.00000	2.2186	1.00000	2.2200	0.00000	0.0014

- Ybus Admittance Matrix Results:

	Bus 1	Bus 2	Bus 3	Bus 4	Bus 5	Bus 6	Bus 7
Bus 1 PY	1.46 -14.64j	-1.46 + 14.64j					
Bus 1 PW	1.46 - j14.65	-1.46 + j14.65					
Bus 2 PY	-1.46 +14.64j	23.59 -155.58j	-6.31 +40.33j	-15.80 +100.84j			
Bus 2 PW	-1.46 + j14.65	23.59 - j155.58	-6.32 + j40.33	-15.81 +j100.85			
Bus 3 PY		-6.31 + 40.33j	14.21 -90.43j		-7.89 + 50.41j		
Bus 3 PW		-6.32 + j40.33	14.21 - j90.43		-7.90 + j50.42		
Bus 4 PY		-15.80 +100.84j		42.14 -268.67j	-10.52 + 67.22j	-15.80 +100.84j	
Bus 4 PW		-15.81 +j100.85		42.14 - j268.67	-10.53 + j67.22	-15.81 +j100.85	
Bus 5 PY			-7.89 +50.41j	-10.52 + 67.22j	22.93 -145.95j	-4.51 + 28.80j	
Bus 5 PW			-7.90 + j50.42	-10.53 + j67.22	22.94 -j145.95	-4.51 + j28.81	
Bus 6 PY				-15.80 +100.84j	-4.51 + 28.80j	21.9 -148.31j	-1.57 +18.97j
Bus 6 PW				-15.81 +j100.85	-4.51 + j28.81	21.90 - j148.31	-1.58 + j18.97
Bus 7 PY						-1.57 + 18.97j	1.57 -18.97j
Bus 7 PW						-1.58 + j18.97	1.58 - j18.97

- **Fault Analysis Test Results:** Faulted Bus – Bus 2
 - + I_fault – Python: 11.689 pu
 - + I_fault – PowerWorld: 11.234 pu

V_fault (pu) (Python)	V_fault (pu) (PowerWorld)	Error
0.3619	0.37847	0.0166
0.0000	0.00000	0.0000
0.0260	0.02611	0.0001
0.0544	0.04521	0.0092
0.0528	0.04721	0.0092
0.1121	0.08938	0.0227
0.5270	0.38919	0.1378

As can be seen, there are differences in fault current and fault voltages between Python and PowerWorld. The main reason is Python uses $v_f=1.0$ while PowerWorld uses actual prefault voltages. However, in general, Python implementation is theoretically correct.

- **Jacobian submatrix J1:**

	Bus 2	Bus 3	Bus 4	Bus 5	Bus 6	Bus 7
Bus 2 (PY)	147.73	-38.1	-95.5			
Bus 2 (PW)	147.73	-38.11	-95.50			
Bus 3 (PY)	-37.87	85.03		-47.15		
Bus 3 (PW)	-37.87	85.03		-47.15		
Bus 4 (PY)	-95.41		254.21	-63.33	-95.47	
Bus 4 (PW)	-95.41		254.21	-63.33	-95.47	
Bus 5 (PY)		-47.25	-63.13	137.52	-27.14	
Bus 5 (PW)		-47.25	-63.13	137.52	-27.14	
Bus 6 (PY)			-95.85	-27.33	141.45	-18.26
Bus 6 (PW)			-95.86	-27.34	141.45	-18.26
Bus 7 (PY)					-18.58	18.58

Bus 7 (PW)					-18.59	18.59
-----------------------	--	--	--	--	--------	-------

- **Jacobian submatrix J2:**

	Bus 2	Bus 3	Bus 4	Bus 5	Bus 6
Bus 2 (PY)	22.98	-5.39	-15.10		
Bus 2 (PW)	22.99	-5.39	-15.10		
Bus 3 (PY)	-6.86	12.60		-7.95	
Bus 3 (PW)	-6.87	12.60		-7.95	
Bus 4 (PY)	-15.66		39.91	-9.58	-16.62
Bus 4 (PW)	-15.66		39.91	-9.58	-16.62
Bus 5 (PY)		-7.33	-10.84	21.18	-5.00
Bus 5 (PW)		-7.33	-10.84	21.18	-5.01
Bus 6 (PY)			-14.16	-3.76	21.39
Bus 6 (PW)			-14.16	-3.76	21.39
Bus 7 (PY)					0.43
Bus 7 (PW)					0.43

- **Jacobian submatrix J3:**

	Bus 2	Bus 3	Bus 4	Bus 5	Bus 6	Bus 7
Bus 2 (PY)	-22.40	5.21	14.67			
Bus 2 (PW)	-22.40	5.21	14.67			
Bus 3 (PY)	6.69	-14.38		7.70		
Bus 3 (PW)	6.69	-14.39		7.70		

Bus 4 (PY)	15.25		-40.76	9.27	16.23	
Bus 4 (PW)	15.26		-40.76	9.27	16.23	
Bus 5 (PY)		7.08	10.53	-22.51	4.89	
Bus 5 (PW)		7.09	10.53	-22.51	4.89	
Bus 6 (PY)			13.75	3.64	-20.89	3.49
Bus 6 (PW)			13.76	3.64	-20.89	3.49

- **Jacobian submatrix J4:**

	Bus 2	Bus 3	Bus 4	Bus 5	Bus 6
Bus 2 (PY)	151.61	-39.41	-98.32		
Bus 2 (PW)	151.61	-39.41	-98.32		
Bus 3 (PY)	-38.87	86.91		-48.69	
Bus 3 (PW)	-38.87	86.91		-48.69	
Bus 4 (PY)	-97.91		260.26	-65.39	-97.75
Bus 4 (PW)	-97.91		260.26	-65.40	-97.75
Bus 5 (PY)		-48.87	-64.99	140.67	-27.79
Bus 5 (PW)		-48.87	-64.99	140.67	-27.79
Bus 6 (PY)			-98.68	-28.23	144.84
Bus 6 (PW)			-98.68	-28.23	144.84

- **Sensitivity Analysis Results:** Compare the results between Python Sensitivity Analysis and Sensitivity Analysis in Power World – Bus 3 was selected:

	P sensitivity	Q sensitivity	δP sensitivity	δQ sensitivity
Bus 2 (PY)	0.0083	0.0377	0.0712	-0.0004

Bus 2 (PW)	0.00006708	0.00037609	0.00071045	0.00001268
Bus 3 (PY)	0.0107	0.0518	0.0876	-0.0008
Bus 3 (PW)	0.000106724	0.00051755	0.00087609	-0.00000831
Bus 4 (PY)	0.0081	0.0374	0.0752	-0.0000
Bus 4 (PW)	0.00006930	0.00037405	0.00075003	0.00000837
Bus 5 (PY)	0.0090	0.0420	0.0798	0.0001
Bus 5 (PW)	0.00008486	0.00041988	0.00079637	0.00000310
Bus 6 (PY)	0.0073	0.0336	0.0761	-0.0004
Bus 6 (PW)	0.00005389	0.00033597	0.00075630	0.00000597
Bus 7 (PY)	-	-	0.0760	-0.0012
Bus 7 (PW)	-	-	0.00074194	-

We can see the differences in the results between Python simulator and PowerWorld.

- The first reason is PowerWorld calculate sensitivity in 1MVA base, while Python Simulator use 100MVA base. This is why the values in Python simulator are 100 times larger than those in PowerWorld.
- The second reason is PowerWorld uses a different approach to calculate the sensitivity, as we can look at P and δQ sensitivity results. However, Q and δP have the same values for sensitivity results.

With these reasons, combine with the same results of Jacobian matrix and inverse Jacobian matrix between Python Simulator and PowerWorld, we can say that the Simulator is working correctly.

- **Small Load Perturbation:** In this case, a small perturbation of 0.02pu of real power was applied to the load at Bus 3. The results are compared between PowerWorld and Simulator's Sensitivity Analysis.

	V_new	δ_new
Bus 1 (PY)	1.0000	0.0000
Bus 1 (PW)	1.00000	0.0000
Error	0.000	0.000
Bus 2 (PY)	0.9746	-4.3212
Bus 2 (PW)	0.9743	-4.48
Error	0.0003	0.1588
Bus 3 (PY)	0.9670	-5.4164

Bus 3 (PW)	0.9666	-5.62
Error	0.0004	0.2036
Bus 4 (PY)	0.9715	-4.4927
Bus 4 (PW)	0.9712	-4.66
Error	0.0003	0.1673
Bus 5 (PY)	0.9686	-5.0566
Bus 5 (PW)	0.9682	-5.24
Error	0.0004	0.1834
Bus 6 (PY)	0.9768	-3.7496
Bus 6 (PW)	0.9765	-3.92
Error	0.0003	0.1704
Bus 7 (PY)	1.0000	2.3057
Bus 7 (PW)	1.00002	2.13
Error	0.00002	0.1757

It can be seen that the errors between Python Simulator and PowerWorld are small, with around 0.0003 pu and 0.0004 pu for voltages, and 0.15 deg to 0.2 deg for angles.

6. Conclusion

This project successfully implemented a complete Python-based power system analysis simulator from scratch, covering three major areas of power system analysis: Newton-Raphson power flow, three-phase symmetrical fault analysis, and linear voltage sensitivity analysis.

The Newton-Raphson power flow solver converges in 4 iterations for the 7-bus test system, producing results that match PowerWorld Simulator. The nodal admittance matrix (Y_{bus}) matches PowerWorld exactly for all 49 entries of the 7×7 matrix, confirming that the network assembly and stamping process is correctly implemented.

The Jacobian matrix computed at the converged operating point matches PowerWorld's Jacobian exactly across all four submatrices (J_1 , J_2 , J_3 , J_4).

The three-phase symmetrical fault analysis correctly computes fault currents and post-fault bus voltages using the Zbus matrix method. The faulted bus voltage collapses to 0.0 pu in all cases as expected. Small differences in fault current magnitude compared to PowerWorld (approximately 0.37% at Bus 1) are attributable to the flat-start prefault voltage assumption ($V_f = 1.0$ pu) used by the Python Simulator, versus the actual converged prefault voltages used by PowerWorld. Both approaches are theoretically valid.

For Sensitivity Analysis, Python Simulator and PowerWorld use different approaches to compute sensitivity, so the results may differ slightly. However, since both the Jacobian matrix and its inverse match exactly between the two simulators, the Python simulator is confirmed to be working correctly. This was further validated by applying a small load perturbation of 0.02 pu at Bus 3 and comparing the post-perturbation voltages and angles between Python and PowerWorld, showing voltage errors below 0.0004 pu and angle errors below 0.2 degrees.

Overall, the Python power system simulator produces results that are in strong agreement with PowerWorld Simulator across all three analysis modes. The small differences attributable to known modeling assumptions rather than implementation errors. The simulator provides a solid foundation for further extensions for future studies.